3202 vo N 8]LC .sc[5v46890.4012:vi Xra

ROFORMER：具有旋转位置嵌入的增强型Transformer

苏剑林 追一科技有限公司 深圳 bojonesu@wezhuiyi.com

深圳追一科技有限公司 julianlu@wezhuiyi.com

盛丰攀 追一科技有限公司 深圳 nickpan@wezhuiyi.com

艾哈迈德·穆尔塔达 追一科技有限公司 深圳 mengjiayi@wezhuiyi.com

博文 追一科技有限公司 深圳 brucewen@wezhuiyi.com

刘云峰 追一科技有限公司 深圳 glenliu@wezhuiyi.com

2023年11月9日

摘要

位置编码最近在Transformer架构中显示出有效性。它为序列中不同位置元素之间的依赖关系建模提供了有价值的监督。在本文中，我们首先研究了将位置信息整合到基于Transformer的语言模型学习过程中的各种方法。然后，我们提出了一种名为旋转位置嵌入（RoPE）的新方法，以有效利用位置信息。具体而言，所提出的RoPE使用旋转矩阵编码绝对位置，同时将显式的相对位置依赖关系融入自注意力公式中。值得注意的是，RoPE具有一些有价值的特性，包括序列长度的灵活性、随相对距离增加而衰减的token间依赖关系，以及为线性自注意力配备相对位置编码的能力。最后，我们在多个长文本分类基准数据集上评估了带有旋转位置嵌入的增强型Transformer，也称为RoFormer。我们的实验表明，它始终优于其他替代方案。此外，我们提供了理论分析来解释一些实验结果。RoFormer已被集成到Huggingface中：https://huggingface.co/docs/transformers/model_doc/roformer。

Keywords 预训练语言模型 · 位置信息编码 · 预训练 · 自然语言处理。

1 引言

单词的顺序对于自然语言理解具有重要价值。基于循环神经网络（RRNs）的模型通过沿时间维度递归计算隐藏状态来编码词元的顺序。基于卷积神经网络（CNNs）的模型（CNNs）Gehring et al. [2017] 通常被认为与位置无关，但最近的工作 Islam et al. [2020] 表明，常用的填充操作可以隐式地学习位置信息。最近，建立在 Transformer Vaswani et al. [2017] 之上的预训练语言模型（PLMs）在各种自然语言处理（NLP）任务上取得了最先进的性能，包括上下文表示学习 Devlin et al. [2019]、机器翻译 Vaswani et al. [2017] 以及语言建模 Radford et al. [2019] 等。与基于 RRNs 和 CNNs 的模型不同，PLMs 利用自注意力机制来语义化地捕获给定语料的上下文表示。因此，PLMs 在并行化方面相较于 RNNs 取得了显著改进，并且与 CNNs[1] 相比提高了对更长词元间关系的建模能力。

[1]A stack of multiple CNN layers can also capture longer intra-token relation, here we only consider single layer setting.

值得注意的是，当前预训练语言模型的自注意力架构已被证明具有位置无关性Yun等人[2020]。遵循这一主张，多种方法被提出以将位置信息编码到学习过程中。一方面，通过预定义函数生成的绝对位置编码Vaswani等人[2017]被添加到上下文表示中，而可训练的绝对位置编码Gehring等人[2017]、Devlin等人[2019]、Lan等人[2020]、Clark等人[2020]、Radford等人[2019]、Radford和Narasimhan[2018]也被采用。另一方面，先前的工作Parikh等人[2016]、Shaw等人[2018]、Huang等人[2018]、Dai等人[2019]、Yang等人[2019]、Raffel等人[2020]、Ke等人[2020]、He等人[2020]、Huang等人[2020]聚焦于相对位置编码，通常将相对位置信息编码到注意力机制中。除这些方法外，Liu等人[2020]的作者提出从神经常微分方程Chen等人[2018a]的角度建模位置编码的依赖性，Wang等人[2020]的作者提出在复空间中建模位置信息。尽管这些方法有效，但它们通常将位置信息添加到上下文表示中，因而使其不适用于线性自注意力架构。

在本文中，我们介绍了一种新颖的方法，即旋转位置嵌入（RoPE），以将位置信息利用到PLMS的学习过程中。具体而言，RoPE使用旋转矩阵对绝对位置进行编码，同时将显式的相对位置依赖性纳入自注意力公式中。请注意，所提出的RoPE通过有价值的特性优先于现有方法，包括序列长度的灵活性、随着相对距离增加而衰减的令牌间依赖性，以及为线性自注意力配备相对位置编码的能力。在各种长文本分类基准数据集上的实验结果表明，采用旋转位置嵌入的增强型Transformer，即RoFormer，与基线替代方案相比可以提供更好的性能，从而证明了所提出的RoPE的有效性。

简而言之，我们的贡献有以下三个方面:

- 我们研究了现有的相对位置编码方法，并发现它们大多基于将位置编码加到上下文表示中的分解思想。我们引入了一种新颖的方法，即旋转位置嵌入（RoPE），以将位置信息利用到PLMS的学习过程中。其核心思想是通过将上下文表示与具有清晰理论解释的旋转矩阵相乘来编码相对位置。

- 我们研究了RoPE的性质，并表明它随着相对距离的增加而衰减，这是自然语言编码所需要的。我们善意地指出，以往基于相对位置编码的方法与线性自注意力不兼容。

- 我们在各种长文本基准数据集上评估了所提出的RoFormer。我们的实验表明，与替代方案相比，它始终取得更好的性能。一些使用预训练语言模型的实验可在GitHub上获取：https://github.com/ZhuiyiTechnology/roformer。

本文的其余部分组织如下。我们在第(2)节中建立自注意力架构中位置编码问题的形式化描述，并回顾先前的工作。然后，我们在第(3)节中描述旋转位置编码(RoPE)并研究其性质。我们在第(4)节中报告实验。最后，我们在第(5)节中总结本文。

2 背景与相关工作

2.1 预备

令 $\mathbb{S}_N = \{w_i\}_{i=1}^N$ 为 N 个输入 token 的序列，其中 w_i 是第 i^{th} 个元素。$\mathbb{S}_N$ 对应的词嵌入记为 $\mathbb{E}_N = \{\boldsymbol{x}_i\}_{i=1}^N$，其中 $\boldsymbol{x}_i \in \mathbb{R}^d$ 是 token w_i 的 d 维词嵌入向量，不包含位置信息。自注意力首先将位置信息融入词嵌入，并将其转换为查询、键和值表示。

$$\begin{aligned} \boldsymbol{q}_m &= f_q(\boldsymbol{x}_m, m) \\ \boldsymbol{k}_n &= f_k(\boldsymbol{x}_n, n) \\ \boldsymbol{v}_n &= f_v(\boldsymbol{x}_n, n), \end{aligned} \tag{1}$$

其中 $\boldsymbol{q}_m, \boldsymbol{k}_n$ 和 $\boldsymbol{v}_n$ 分别通过 f_q, f_k 和 f_v 结合了 m^{th} 和 n^{th} 位置。然后使用查询值和键值来计算注意力权重，而输出则计算为对值的加权和

表示。

$$
\begin{aligned}
a_{m,n} &= \frac{\exp(\frac{\boldsymbol{q}_m^\intercal \boldsymbol{k}_n}{\sqrt{d}})}{\sum_{j=1}^{N} \exp(\frac{\boldsymbol{q}_m^\intercal \boldsymbol{k}_j}{\sqrt{d}})} \\
\mathbf{o}_m &= \sum_{n=1}^{N} a_{m,n} \boldsymbol{v}_n
\end{aligned}
\tag{2}
$$

现有的基于Transformer的位置编码方法主要集中于选择合适的函数来构成公式(1)。

2.2 绝对位置嵌入

公式（1）的一个典型选择是

$$
f_{t:t\in\{q,k,v\}}(\boldsymbol{x}_i, i) := \boldsymbol{W}_{t:t\in\{q,k,v\}}(\boldsymbol{x}_i + \boldsymbol{p}_i), \tag{3}
$$

其中 $\boldsymbol{p}_i \in \mathbb{R}^d$ 是一个依赖于 token $\boldsymbol{x}_i$ 位置的 d 维向量。先前的工作 Devlin et al. [2019]、Lan et al. [2020]、Clark et al. [2020]、Radford et al. [2019]、Radford and Narasimhan [2018] 引入了使用一组可训练向量 $\boldsymbol{p}_i \in \{\boldsymbol{p}_t\}_{t=1}^{L}$，其中 L 是最大序列长度。Vaswani et al. [2017] 的作者提出使用正弦函数生成 $\boldsymbol{p}_i$。

$$
\begin{cases}
\boldsymbol{p}_{i,2t} &= \sin(k/10000^{2t/d}) \\
\boldsymbol{p}_{i,2t+1} &= \cos(k/10000^{2t/d})
\end{cases}
\tag{4}
$$

其中 $\boldsymbol{p}_{i,2t}$ 是 d 维向量 $\boldsymbol{p}_i$ 的第 $2t^{th}$ 个元素。在下一节中，我们将展示，从正弦函数的角度来看，我们提出的 RoPE 与这一直觉相关。然而，RoPE 并非直接将位置加到上下文表示中，而是提出通过与正弦函数相乘来融入相对位置信息。

2.3 相对位置嵌入

Shaw等人[2018]的作者对公式(1)应用了如下不同的设置：

$$
\begin{aligned}
f_q(\boldsymbol{x}_m) &:= \boldsymbol{W}_q \boldsymbol{x}_m \\
f_k(\boldsymbol{x}_n, n) &:= \boldsymbol{W}_k(\boldsymbol{x}_n + \tilde{\boldsymbol{p}}_r^k) \\
f_v(\boldsymbol{x}_n, n) &:= \boldsymbol{W}_v(\boldsymbol{x}_n + \tilde{\boldsymbol{p}}_r^v)
\end{aligned}
\tag{5}
$$

其中 $\tilde{\boldsymbol{p}}_r^k, \tilde{\boldsymbol{p}}_r^v \in \mathbb{R}^d$ 是可训练的相對位置嵌入。注意 $r = \text{clip}(m-n, r_{\text{min}}, r_{\text{max}})$ 表示位置 m 和 n 之間的相對距離。他們對相對距離進行了裁剪，假設精確的相對位置信息在超過一定距離後不再有用。保持公式 (3) 的形式，作者 Dai 等人 [2019] 提出將公式 (2) 中的 $\boldsymbol{q}_m^\intercal \boldsymbol{k}_n$ 分解為

$$
\boldsymbol{q}_m^\intercal \boldsymbol{k}_n = \boldsymbol{x}_m^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{x}_n + \boldsymbol{x}_m^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{p}_n + \boldsymbol{p}_m^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{x}_n + \boldsymbol{p}_m^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{p}_n, \tag{6}
$$

关键思想是将绝对位置嵌入$\boldsymbol{p}_n$替换为其正弦编码的相对应版本$\tilde{\boldsymbol{p}}_{m-n}$，同时将第三项和第四项中的绝对位置$\boldsymbol{p}_m$替换为两个与查询位置无关的可训练向量$\mathbf{u}$和$\mathbf{v}$。此外，对于基于内容和基于位置的关键向量$\boldsymbol{x}_n$和$\boldsymbol{p}_n$，$\boldsymbol{W}_k$被分别区分为$\boldsymbol{W}_k$和$\widetilde{\boldsymbol{W}}_k$，从而得到：

$$
\boldsymbol{q}_m^\intercal \boldsymbol{k}_n = \boldsymbol{x}_m^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{x}_n + \boldsymbol{x}_m^\intercal \boldsymbol{W}_q^\intercal \widetilde{\boldsymbol{W}}_k \tilde{\boldsymbol{p}}_{m-n} + \mathbf{u}^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{x}_n + \mathbf{v}^\intercal \boldsymbol{W}_q^\intercal \widetilde{\boldsymbol{W}}_k \tilde{\boldsymbol{p}}_{m-n} \tag{7}
$$

值得注意的是，值项中的位置信息通过设置 $f_v(\boldsymbol{x}_j) := \boldsymbol{W}_v \boldsymbol{x}_j$ 被移除。后续工作 Raffel 等人 [2020]、He 等人 [2020]、Ke 等人 [2020]、Huang 等人 [2020] 遵循了这些设置，仅将相对位置信息编码到注意力权重中。然而，Raffel 等人 [2020] 的作者将公式 (6) 修改为：

$$
\boldsymbol{q}_m^\intercal \boldsymbol{k}_n = \boldsymbol{x}_m^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{x}_n + b_{i,j} \tag{8}
$$

其中 $b_{i,j}$ 是一个可训练的偏置。Ke 等人 [2020] 的作者研究了公式 (6) 的中间两项，发现绝对位置与词之间的相关性很小。Raffel 等人 [2020] 的作者提出使用不同的投影矩阵对一对词或位置进行建模。

$$
\boldsymbol{q}_m^\intercal \boldsymbol{k}_n = \boldsymbol{x}_m^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{x}_n + \boldsymbol{p}_m^\intercal \mathbf{U}_q^\intercal \mathbf{U}_k \boldsymbol{p}_n + b_{i,j} \tag{9}
$$

He 等人 [2020] 的作者认为，两个标记的相对位置只能通过公式 (6) 的中间两项来完全建模。因此，绝对位置嵌入 $\boldsymbol{p}_m$ 和 $\boldsymbol{p}_n$ 被简单地替换为相对位置嵌入 $\tilde{\boldsymbol{p}}_{m-n}$：

$$\boldsymbol{q}_m^\intercal \boldsymbol{k}_n = \boldsymbol{x}_m^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{x}_n + \boldsymbol{x}_m^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \tilde{\boldsymbol{p}}_{m-n} + \tilde{\boldsymbol{p}}_{m-n}^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{x}_n \tag{10}$$

Radford和Narasimhan[2018]对相对位置嵌入的四种变体进行了比较，结果表明，与公式(10)相似的变体在其他三种变体中最为高效。一般来说，所有这些方法都试图基于公式(2)中自注意力设置下公式(3)的分解来修改公式(6)，这一设置最初由Vaswani等人[2017]提出。它们通常引入直接将位置信息添加到上下文表示中。不同的是，我们的方法旨在在某些约束下从公式(1)推导出相对位置编码。接下来，我们通过将相对位置信息与上下文表示的旋转相结合，表明所推导的方法更具可解释性。

3 提出的方法

在本节中，我们讨论所提出的旋转位置嵌入（RoPE）。我们首先在第（3.1）节中形式化相对位置编码问题，然后在第（3.2）节中推导 RoPE，并在第（3.3）节中研究其性质。

3.1 公式化

基于Transformer的语言建模通常通过自注意力机制利用单个词元的位置信息。如公式（2）所示，$\boldsymbol{q}_m^\intercal \boldsymbol{k}_n$通常能够实现不同位置词元之间的知识传递。为了融入相对位置信息，我们要求查询$\boldsymbol{q}_m$和键$\boldsymbol{k}_n$的内积由一个函数g来构建，该函数仅以词嵌入$\boldsymbol{x}_m$、$\boldsymbol{x}_n$及其相对位置$m-n$作为输入变量。换言之，我们希望内积仅以相对形式编码位置信息：

$$\langle f_q(\boldsymbol{x}_m, m), f_k(\boldsymbol{x}_n, n)\rangle = g(\boldsymbol{x}_m, \boldsymbol{x}_n, m-n). \tag{11}$$

最终目标是找到一种等效的编码机制来求解函数 $f_q(\boldsymbol{x}_m, m)$ 和 $f_k(\boldsymbol{x}_n, n)$，以符合上述关系。

3.2 旋转位置嵌入

3.2.1 二维情形

我们从维度 $d=$ 为2的简单情形开始。在这些设定下，我们利用二维平面上向量的几何性质及其复数形式来证明（更多细节参见第(3.4.1)节），我们的公式方程(11)的一个解是：

$$\begin{aligned} f_q(\boldsymbol{x}_m, m) &= (\boldsymbol{W}_q \boldsymbol{x}_m) e^{im\theta} \\ f_k(\boldsymbol{x}_n, n) &= (\boldsymbol{W}_k \boldsymbol{x}_n) e^{in\theta} \\ g(\boldsymbol{x}_m, \boldsymbol{x}_n, m-n) &= \mathrm{Re}[(\boldsymbol{W}_q \boldsymbol{x}_m)(\boldsymbol{W}_k \boldsymbol{x}_n)^* e^{i(m-n)\theta}] \end{aligned} \tag{12}$$

其中 Re[·] 是复数的实部，$(\boldsymbol{W}_k \boldsymbol{x}_n)^*$ 表示 $(\boldsymbol{W}_k \boldsymbol{x}_n)$ 的共轭复数。$\theta \in \mathbb{R}$ 是预设的非零常数。我们可以进一步将 $f_{\{q,k\}}$ 写成乘法矩阵形式：

$$f_{\{q,k\}}(\boldsymbol{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W_{\{q,k\}}^{(11)} & W_{\{q,k\}}^{(12)} \\ W_{\{q,k\}}^{(21)} & W_{\{q,k\}}^{(22)} \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix} \tag{13}$$

其中 $(x_m^{(1)}, x_m^{(2)})$ 是在二维坐标中表示的 $\boldsymbol{x}_m$。类似地，g 可以被视为一个矩阵，从而能够在二维情况下求解第(3.1)节中的公式。具体而言，引入相对位置嵌入是很直接的：只需将仿射变换后的词嵌入向量按其位置索引的角度倍数进行旋转，从而解释 *Rotary Position Embedding* 背后的直觉。

3.2.2 一般形式

为了将我们在二维中的结果推广到任何$\boldsymbol{x}_i \in \mathbb{R}^d$，其中$d$为偶数，我们将d维空间划分为$d/2$个子空间，并利用内积的线性性质将它们组合起来，将$f_{\{q,k\}}$转化为：

$$f_{\{q,k\}}(\boldsymbol{x}_m, m) = \boldsymbol{R}^d_{\Theta,m}\boldsymbol{W}_{\{q,k\}}\boldsymbol{x}_m \tag{14}$$

在哪里

$$\boldsymbol{R}^d_{\Theta,m} = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix} \tag{15}$$

是带有预定义参数 $\Theta = \{\theta_i = 10000^{-2(i-1)/d}, i \in [1, 2, ..., d/2]\}$ 的旋转矩阵。RoPE 的图示见图 (1)。将我们的 RoPE 应用于公式 (2) 中的自注意力，我们得到：

$$\boldsymbol{q}_m^\intercal \boldsymbol{k}_n = (\boldsymbol{R}^d_{\Theta,m}\boldsymbol{W}_q\boldsymbol{x}_m)^\intercal(\boldsymbol{R}^d_{\Theta,n}\boldsymbol{W}_k\boldsymbol{x}_n) = \boldsymbol{x}^\intercal\boldsymbol{W}_q R^d_{\Theta,n-m}\boldsymbol{W}_k\boldsymbol{x}_n \tag{16}$$

其中 $\boldsymbol{R}^d_{\Theta,n-m} = (\boldsymbol{R}^d_{\Theta,m})^\intercal\boldsymbol{R}^d_{\Theta,n}$。注意 $\boldsymbol{R}^d_\Theta$ 是一个正交矩阵，这保证了位置信息编码过程中的稳定性。此外，由于 R^d_Θ 的稀疏性，直接按照公式 (16) 进行矩阵乘法在计算上并不高效；我们在理论解释中提供了另一种实现方式。

与先前工作中所采用的位置嵌入方法的加性特性（即公式(3)至(10)）不同，我们的方法是乘性的。此外，RoPE通过旋转矩阵乘积自然地融入了相对位置信息，而不是在自注意力应用时改变加性位置编码展开式中的项。

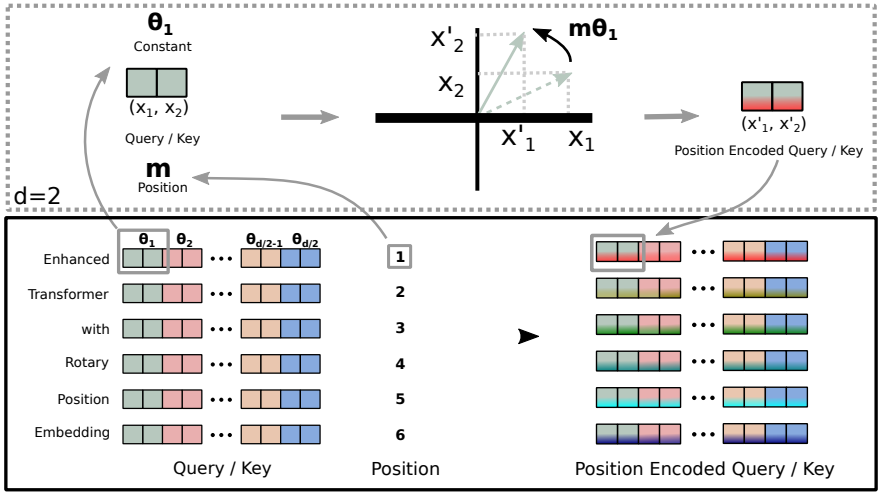


图1：旋转位置嵌入（RoPE）的实现。

3.3 RoPE的性质

长期衰减：遵循Vaswani等人[2017]的做法，我们设置$\theta_i = 10000^{-2i/d}$。可以证明，这一设置提供了长期衰减特性（更多细节请参阅第(3.4.3)节），这意味着当相对位置增加时，内积将会衰减。这一特性与直觉相符，即相对距离较远的一对词元应当具有较少的联系。

带线性注意力的RoPE：自注意力可以重写为更一般的形式。

$$\text{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V})_m = \frac{\sum_{n=1}^{N}\text{sim}(\boldsymbol{q}_m,\boldsymbol{k}_n)\boldsymbol{v}_n}{\sum_{n=1}^{N}\text{sim}(\boldsymbol{q}_m,\boldsymbol{k}_n)}. \tag{17}$$

原始自注意力选择sim($\boldsymbol{q}_m,\boldsymbol{k}_n$) = exp($\boldsymbol{q}_m^\intercal\boldsymbol{k}_n/\sqrt{d}$)。注意，原始自注意力需要为每一对token计算query和key的内积，其复杂度为二次复杂度$\mathbb{O}(N^2)$。遵循Katharopoulos等人[2020]的工作，线性注意力将公式(17)重新表述为

$$\text{Attention}(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V})_m = \frac{\sum_{n=1}^{N}\phi(\boldsymbol{q}_m)^\intercal\varphi(\boldsymbol{k}_n)\boldsymbol{v}_n}{\sum_{n=1}^{N}\phi(\boldsymbol{q}_m)^\intercal\varphi(\boldsymbol{k}_n)}, \tag{18}$$

其中 $\phi(\cdot),\varphi(\cdot)$ 通常是非负函数。Katharopoulos 等人 [2020] 的作者提出了 $\phi(x)=\varphi(x)=\text{elu}(x)+1$，并首先利用矩阵乘法的结合性质计算键与值之间的乘法。Shen 等人 [2021] 使用 softmax 函数在内积之前分别对查询和键进行归一化，这等价于 $\phi(\boldsymbol{q}_i)=\text{softmax}(\boldsymbol{q}_i)$ 和 $\phi(\boldsymbol{k}_j)=\exp(\boldsymbol{k}_j)$。关于线性注意力的更多细节，我们鼓励读者参阅原始论文。在本节中，我们重点讨论将 RoPE 与公式 (18) 相结合。由于 RoPE 通过旋转注入位置信息，而旋转保持隐藏表示的范数不变，我们可以通过将旋转矩阵与非负函数的输出相乘，将 RoPE 与线性注意力相结合。

$$\text{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V})_m = \frac{\sum_{n=1}^{N}\left(\boldsymbol{R}^d_{\Theta,m}\phi(\boldsymbol{q}_m)\right)^\intercal\left(\boldsymbol{R}^d_{\Theta,n}\varphi(\boldsymbol{k}_n)\right)\boldsymbol{v}_n}{\sum_{n=1}^{N}\phi(\boldsymbol{q}_m)^\intercal\varphi(\boldsymbol{k}_n)}. \tag{19}$$

值得注意的是，我们保持分母不变以避免除以零的风险，而分子中的求和可能包含负项。尽管公式（19）中每个值 $\boldsymbol{v}_i$ 的权重并非严格按概率归一化，但我们认为该计算仍能建模值的重要性。

3.4 理论解释

3.4.1 二维下RoPE的推导

在 $d=2$ 的情况下，我们考虑两个词嵌入向量 $\boldsymbol{x}_q$、$\boldsymbol{x}_k$ 分别对应查询和键，以及它们的位置 m 和 n。根据式 (1)，它们的位置编码对应项为：

$$\begin{aligned}\boldsymbol{q}_m &= f_q(\boldsymbol{x}_q,m),\\ \boldsymbol{k}_n &= f_k(\boldsymbol{x}_k,n),\end{aligned} \tag{20}$$

其中 $\boldsymbol{q}_m$ 和 $\boldsymbol{k}_n$ 的下标表示编码后的位置信息。假设存在一个函数 g，它定义了由 $f_{\{q,k\}}$ 生成的向量之间的内积：

$$\boldsymbol{q}_m^\intercal\boldsymbol{k}_n = \langle f_q(\boldsymbol{x}_m,m), f_k(\boldsymbol{x}_n,n)\rangle = g(\boldsymbol{x}_m,\boldsymbol{x}_n,n-m), \tag{21}$$

我们进一步要求满足以下初始条件：

$$\begin{aligned}\boldsymbol{q} &= f_q(\boldsymbol{x}_q,0),\\ \boldsymbol{k} &= f_k(\boldsymbol{x}_k,0),\end{aligned} \tag{22}$$

这可以理解为编码了空位置信息的向量。给定这些设置，我们尝试寻找f_q、f_k的解。首先，我们利用二维中向量的几何意义及其复数对应物，将方程（20）和（21）中的函数分解为：

$$\begin{aligned}f_q(\boldsymbol{x}_q,m) &= R_q(\boldsymbol{x}_q,m)e^{i\Theta_q(\boldsymbol{x}_q,m)},\\ f_k(\boldsymbol{x}_k,n) &= R_k(\boldsymbol{x}_k,n)e^{i\Theta_k(\boldsymbol{x}_k,n)},\\ g(\boldsymbol{x}_q,\boldsymbol{x}_k,n-m) &= R_g(\boldsymbol{x}_q,\boldsymbol{x}_k,n-m)e^{i\Theta_g(\boldsymbol{x}_q,\boldsymbol{x}_k,n-m)},\end{aligned} \tag{23}$$

其中 R_f、R_g 和 Θ_f、Θ_g 分别是 $f_{\{q,k\}}$ 和 g 的径向分量和角向分量。将它们代入方程（21），我们得到关系：

$$\begin{aligned}R_q(\boldsymbol{x}_q,m)R_k(\boldsymbol{x}_k,n) &= R_g(\boldsymbol{x}_q,\boldsymbol{x}_k,n-m),\\ \Theta_k(\boldsymbol{x}_k,n)-\Theta_q(\boldsymbol{x}_q,m) &= \Theta_g(\boldsymbol{x}_q,\boldsymbol{x}_k,n-m),\end{aligned} \tag{24}$$

相应的初始条件为:

$$\begin{aligned}\boldsymbol{q} &= \|\boldsymbol{q}\|e^{i\theta_q} = R_q(\boldsymbol{x}_q, 0)e^{i\Theta_q(\boldsymbol{x}_q,0)},\\ \boldsymbol{k} &= \|\boldsymbol{k}\|e^{i\theta_k} = R_k(\boldsymbol{x}_k, 0)e^{i\Theta_k(\boldsymbol{x}_k,0)},\end{aligned} \tag{25}$$

其中 $\|\boldsymbol{q}\|$、$\|\boldsymbol{k}\|$ 和 θ_q、θ_k 分别是 $\boldsymbol{q}$ 和 $\boldsymbol{k}$ 在二维平面上的径向和角向部分。
接下来，我们在方程(24)中设置$m = n$，并考虑方程(25)中的初始条件:

$$R_q(\boldsymbol{x}_q, m)R_k(\boldsymbol{x}_k, m) = R_g(\boldsymbol{x}_q, \boldsymbol{x}_k, 0) = R_q(\boldsymbol{x}_q, 0)R_k(\boldsymbol{x}_k, 0) = \|\boldsymbol{q}\|\|\boldsymbol{k}\|, \tag{26a}$$

$$\Theta_k(\boldsymbol{x}_k, m) - \Theta_q(\boldsymbol{x}_q, m) = \Theta_g(\boldsymbol{x}_q, \boldsymbol{x}_k, 0) = \Theta_k(\boldsymbol{x}_k, 0) - \Theta_q(\boldsymbol{x}_q, 0) = \theta_k - \theta_q. \tag{26b}$$

一方面，从式 (26a) 可以直接构造出 R_f 的一个简单解:

$$\begin{aligned}R_q(\boldsymbol{x}_q, m) &= R_q(\boldsymbol{x}_q, 0) = \|\boldsymbol{q}\|\\ R_k(\boldsymbol{x}_k, n) &= R_k(\boldsymbol{x}_k, 0) = \|\boldsymbol{k}\|\\ R_g(\boldsymbol{x}_q, \boldsymbol{x}_k, n-m) &= R_g(\boldsymbol{x}_q, \boldsymbol{x}_k, 0) = \|\boldsymbol{q}\|\|\boldsymbol{k}\|\end{aligned} \tag{27}$$

其中径向函数 R_q、R_k 和 R_g 被解释为与位置信息无关。另一方面，如式 (26b) 所示，
$\Theta_q(\boldsymbol{x}_q, m) - \theta_q = \Theta_k(\boldsymbol{x}_k, m) - \theta_k$ 表明角度函数不依赖于查询和键，我们将其设为 Θ_f: $= \Theta_q = \Theta_k$，而项 $\Theta_f(\boldsymbol{x}_{\{q,k\}}, m) - \theta_{\{q,k\}}$ 是位置 m 的函数，且与词嵌入 $\boldsymbol{x}_{\{q,k\}}$ 无关，我们将其记为 $\phi(m)$，从而得到:

$$\Theta_f(\boldsymbol{x}_{\{q,k\}}, m) = \phi(m) + \theta_{\{q,k\}}, \tag{28}$$

此外，将 $n = m + 1$ 代入方程 (24) 并考虑上述方程，我们可以得到:

$$\phi(m+1) - \phi(m) = \Theta_g(\boldsymbol{x}_q, \boldsymbol{x}_k, 1) + \theta_q - \theta_k, \tag{29}$$

由于RHS是一个与m无关的常数，$\phi(m)$在连续整数输入下产生一个等差数列:

$$\phi(m) = m\theta + \gamma, \tag{30}$$

其中 $\theta, \gamma \in \mathbb{R}$ 是常数，且 θ 非零。总结我们从方程 (27) 到 (30) 的解:

$$\begin{aligned}f_q(\boldsymbol{x}_q, m) &= \|\boldsymbol{q}\|e^{i\theta_q + m\theta + \gamma} = \boldsymbol{q}e^{i(m\theta+\gamma)},\\ f_k(\boldsymbol{x}_k, n) &= \|\boldsymbol{k}\|e^{i\theta_k + n\theta + \gamma} = \boldsymbol{k}e^{i(n\theta+\gamma)}.\end{aligned} \tag{31}$$

注意，我们对公式（22）中的f_q和f_k不施加任何约束，因此$f_q(\boldsymbol{x}_m, 0)$和$f_k(\boldsymbol{x}_n, 0)$可以自由选择。为了使我们的结果与公式（3）具有可比性，我们定义:

$$\begin{aligned}\boldsymbol{q} &= f_q(\boldsymbol{x}_m, 0) = \boldsymbol{W}_q\boldsymbol{x}_n,\\ \boldsymbol{k} &= f_k(\boldsymbol{x}_n, 0) = \boldsymbol{W}_k\boldsymbol{x}_n.\end{aligned} \tag{32}$$

然后，我们只需在最终解的方程（31）中将$\gamma =$设为0:

$$\begin{aligned}f_q(\boldsymbol{x}_m, m) &= (\boldsymbol{W}_q\boldsymbol{x}_m)e^{im\theta},\\ f_k(\boldsymbol{x}_n, n) &= (\boldsymbol{W}_k\boldsymbol{x}_n)e^{in\theta}.\end{aligned} \tag{33}$$

3.4.2 C　旋转矩阵乘法的计算高效实现　　申请

利用公式（15）中$\boldsymbol{R}^d_{\Theta,m}$的稀疏性，$R^d_\Theta$与$\boldsymbol{x} \in \mathbb{R}^d$相乘的一种计算效率更高的实现方式是:

$$\boldsymbol{R}^d_{\Theta,m}\boldsymbol{x} = \begin{pmatrix}x_1\\x_2\\x_3\\x_4\\\vdots\\x_{d-1}\\x_d\end{pmatrix} \otimes \begin{pmatrix}\cos m\theta_1\\\cos m\theta_1\\\cos m\theta_2\\\cos m\theta_2\\\vdots\\\cos m\theta_{d/2}\\\cos m\theta_{d/2}\end{pmatrix} + \begin{pmatrix}-x_2\\x_1\\-x_4\\x_3\\\vdots\\-x_d\\x_{d-1}\end{pmatrix} \otimes \begin{pmatrix}\sin m\theta_1\\\sin m\theta_1\\\sin m\theta_2\\\sin m\theta_2\\\vdots\\\sin m\theta_{d/2}\\\sin m\theta_{d/2}\end{pmatrix} \tag{34}$$

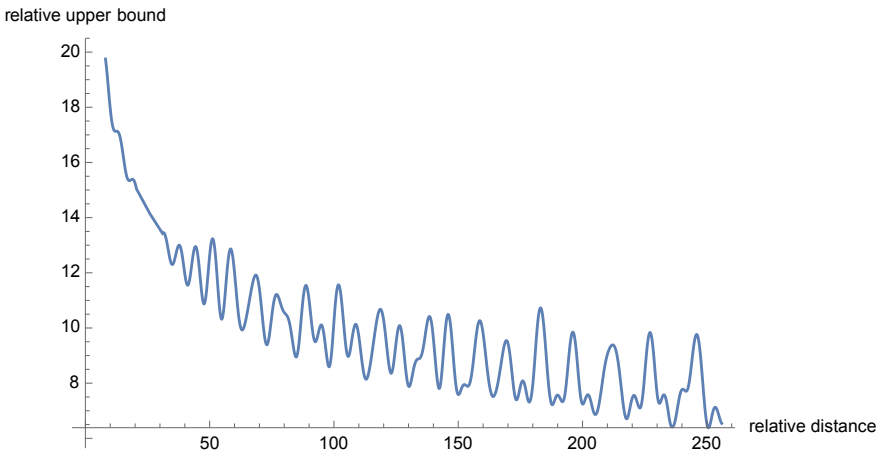


图2：RoPE的长期衰减。

3.4.3 RoPE的长期衰减

我们可以将向量$\boldsymbol{q}=\boldsymbol{W}_q\boldsymbol{x}_m$和$\boldsymbol{k}=\boldsymbol{W}_k\boldsymbol{x}_n$的分量成对分组，式(16)中RoPE的内积可以写成复数乘法。

$$(\boldsymbol{R}^d_{\Theta,m}\boldsymbol{W}_q\boldsymbol{x}_m)^\intercal(\boldsymbol{R}^d_{\Theta,n}\boldsymbol{W}_k\boldsymbol{x}_n)=\mathrm{Re}\left[\sum_{i=0}^{d/2-1}\boldsymbol{q}_{[2i:2i+1]}\boldsymbol{k}^*_{[2i:2i+1]}e^{i(m-n)\theta_i}\right] \tag{35}$$

其中 $\boldsymbol{q}_{[2i:2i+1]}$ 表示 $\boldsymbol{q}$ 的第 $2i^{th}$ 到 $(2i+1)^{th}$ 项。记 $h_i=\boldsymbol{q}_{[2i:2i+1]}\boldsymbol{k}^*_{[2i:2i+1]}$ 和 $S_j=\sum_{i=0}^{j-1}e^{i(m-n)\theta_i}$，并令 $h_{d/2}=0$ 和 $S_0=0$，我们可以使用阿贝尔变换重写该求和

$$\sum_{i=0}^{d/2-1}\boldsymbol{q}_{[2i:2i+1]}\boldsymbol{k}^*_{[2i:2i+1]}e^{i(m-n)\theta_i}=\sum_{i=0}^{d/2-1}h_i(S_{i+1}-S_i)=-\sum_{i=0}^{d/2-1}S_{i+1}(h_{i+1}-h_i). \tag{36}$$

因此，

$$\begin{aligned}\left|\sum_{i=0}^{d/2-1}\boldsymbol{q}_{[2i:2i+1]}\boldsymbol{k}^*_{[2i:2i+1]}e^{i(m-n)\theta_i}\right|&=\left|\sum_{i=0}^{d/2-1}S_{i+1}(h_{i+1}-h_i)\right|\\&\le\sum_{i=0}^{d/2-1}|S_{i+1}||(h_{i+1}-h_i)|\\&\le\left(\max_i|h_{i+1}-h_i|\right)\sum_{i=0}^{d/2-1}|S_{i+1}|\end{aligned} \tag{37}$$

请注意，当设置 $\theta_i=\text{~~10~~}10000^{-2i/d}$ 时，$\frac{1}{d/2}\sum_{i=1}^{d/2}|S_i|$ 的值随着相对距离 $m-n$ 的增加而衰减，如图 (2) 所示。

4 实验与评估

我们按如下方式在各种NLP任务上评估所提出的RoFormer。我们在机器翻译任务第（4.1）节中验证所提出解决方案的性能。然后，在第（4.2）节中，我们在预训练阶段将我们的RoPE实现与BERTDevlin等人[2019]进行比较。基于预训练模型，在第（4.3）节中，我们进一步在GLUE基准Singh等人[2018]的不同下游任务上开展评估。此外，我们在中将所提出的RoPE与PerFormer Choromanski等人[2020]的线性注意力结合进行实验。

表1：在WMT 2014英德翻译任务Bojar等人[2014]上，所提出的RoFormer相比其基线替代方案Vaswani等人[2017]给出了更好的BLEU分数。

Model	BLEU
Transformer-baseVaswani et al. [2017]	27.3
RoFormer	**27.5**

第(4.4)节。最后，第(4.5)节包含了对中文数据的额外测试。所有实验均在两台配备4块V100 GPU的云服务器上运行。

4.1 机器翻译

我们首先展示RoFormer在序列到序列语言翻译任务上的性能。

4.1.1 实验设置

我们选择标准的WMT 2014英德数据集Bojar等人[2014]，该数据集包含约450万个句子对。我们与基于Transformer的基线替代方案Vaswani等人[2017]进行比较。

4.1.2 实现细节

我们对基线模型Vaswani等人[2017]的自注意力层进行了一些修改，以使其学习过程能够使用RoPE。我们复现了英语到德语翻译的设置，基于联合源和目标字节对编码（BPE）Sennrich等人[2015]的37k词汇表。在评估过程中，通过平均最后5个检查点获得单个模型。结果使用束搜索，束大小为4，长度惩罚为0.6。我们在fairseq工具包（MIT许可证）Ott等人[2019]的PyTorch中实现了该实验。我们的模型使用Adam优化器进行优化，使用$\beta_1 = 0.9$、$\beta_2 = 0.98$，学习率从$1e-7$线性增加到$5e-4$，然后按步数的平方根倒数比例衰减。还采用了0.1的标签平滑。我们报告测试集上的BLEUPapineni等人[2002]分数作为最终指标。

4.1.3 结果

我们在相同设置下训练基线模型和我们的RoFormer，并在表（1）中报告结果。可以看出，与基线Transformer相比，我们的模型给出了更好的BLEU分数。

4.2 预训练语言建模

第二个实验是验证我们的提案在学习上下文表示方面的性能。为实现这一目标，我们在预训练阶段用我们的RoPE替换了BERT原有的正弦位置编码。

4.2.1 实验设置

我们使用来自Huggingface Datasets库（Apache License 2.0）的BookCorpus Zhu et al. [2015]和Wikipedia Corpus Foundation [2021]进行预训练。语料库进一步以8：2的比例分为训练集和验证集。我们使用训练过程中的掩码语言建模（MLM）损失值作为评估指标。采用著名的BERT Devlin et al. [2019]作为我们的基线模型。请注意，我们在实验中使用的是bert-base-uncased。

4.2.2 实现细节

对于RoFormer，我们将基线模型自注意力块中的正弦位置编码替换为我们提出的RoPE，并根据公式(16)实现自注意力。我们以批量大小64和最大序列长度512训练BERT和RoFormer各100k步。使用AdamW Loshchilov and Hutter [2017]作为优化器，学习率为1e-5。

4.2.3 结果

预训练期间的MLM损失如图(3)左侧图所示。与原始BERT相比，RoFormer的收敛速度更快。

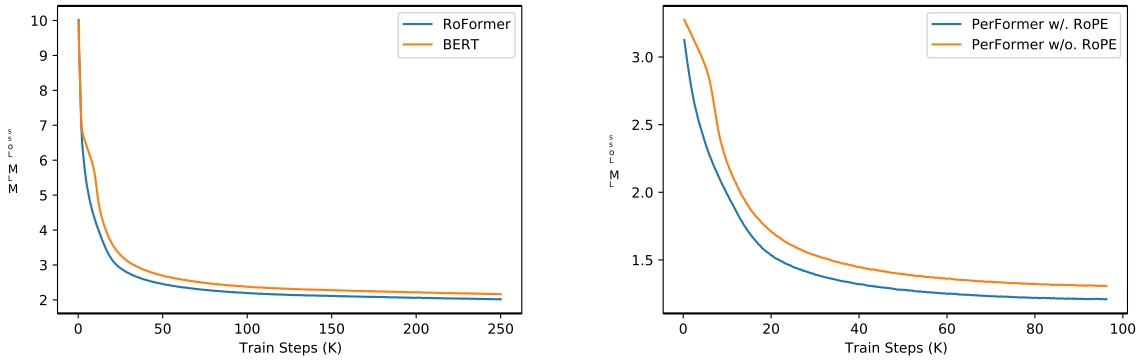


图3：语言建模预训练中RoPE的评估。左：BERT和RoFormer的训练损失。右：使用和不使用RoPE的PerFormer的训练损失。

4.3 在GLUE任务上进行微调

与前文实验一致，我们在各种GLUE任务上微调预训练RoFormer的权重，以评估其在下游NLP任务上的泛化能力。

4.3.1 实验设置

我们考察了来自GLUE的多个数据集，即MRPC Dolan和Brockett [2005]、SST-2 Socher等人 [2013]、QNLI Rajpurkar等人 [2016]、STS-B Al-Natsheh [2017]、QQP Chen等人 [2018b]以及MNLI Williams等人 [2018]。我们使用F1分数作为MRPC和QQP数据集的评估指标，Spearman相关系数用于STS-B，其余数据集则采用准确率作为评估指标。

4.3.2 实现细节

我们使用Huggingface Transformers库（Apache License 2.0）Wolf等人[2020]对上述每个下游任务进行3个epoch的微调，最大序列长度为512，批量大小为32，学习率为2、3、4、5e-5。遵循Devlin等人[2019]的方法，我们报告验证集上的最佳平均结果。

表2：Co　通过在下游G任务上微调来比较RoFormer和BERT　　LEU任务。

Model	MRPC	SST-2	QNLI	STS-B	QQP	MNLI(m/mm)
BERTDevlin et al. [2019]	88.9	93.5	90.5	85.8	71.2	84.6/83.4
RoFormer	**89.5**	90.7	88.0	**87.0**	**86.4**	80.2/79.8

4.3.3 结果

微调任务的评估结果报告在表（2）中。可以看出，RoFormer在六个数据集中的三个上显著优于BERT，且提升幅度相当可观。

4.4 使用RoPE的Performer

Performer Choromanski 等人 [2020] 提出了一种替代注意力机制，即线性注意力，旨在避免随输入序列长度呈二次方增长的计算成本。如第 (3.3) 节所述，所提出的 RoPE 可以很容易地在 PerFormer 模型中实现，以在保持自注意力线性缩放复杂度的同时实现相对位置编码。我们通过语言建模的预训练任务展示了其性能。

4.4.1 实现细节

我们在Enwik8数据集Mahoney [2006]上进行测试，该数据集源自英文维基百科，除了英文文本外，还包括标记、特殊字符及其他语言的文本。我们将RoPE融入具有768维和12个头的12层基于字符的PerFormer中[2]。为了更好地说明RoPE的有效性，我们报告了在相同设置下，即学习率1e-4、批量大小128以及固定最大序列长度1024等条件下，有无RoPE的预训练过程的损失曲线。

4.4.2 结果

如图(3)右侧图所示，将RoPE替换到Performer中，在相同训练步数下可实现快速收敛和更低的损失。这些改进，加上线性复杂度，使Performer更具吸引力。

4.5 中文数据上的评估

除了在英语数据上的实验外，我们还展示了在中文数据上的额外结果。为了验证RoFormer在长文本上的性能，我们在长度超过512个字符的长文档上进行了实验。

4.5.1 实施

在这些实验中，我们对WoBERT Su [2020]进行了一些修改，用我们提出的RoPE替换了绝对位置嵌入。为了与中文其他基于Transformer的预训练模型进行交叉比较，即BERT Devlin等人[2019]、WoBERT Su [2020]和NEZHA Wei等人[2019]，我们在表（3）中列出了它们的分词级别和位置嵌入信息。

表3：我们的RoFormer与其他预训练模型在中文数据上的交叉对比。“abs”和“rel”分别标注绝对位置嵌入和相对位置嵌入。

Model	BERTDevlin et al. [2019]	WoBERTSu [2020]	NEZHAWei et al. [2019]	RoFormer
Tokenization level	char	word	char	word
Position embedding	abs.	abs.	rel.	RoPE

4.5.2 预训练

我们在约34GB的来自中文维基百科、新闻和论坛的数据上预训练RoFormer。预训练分多个阶段进行，期间改变批量大小和最大输入序列长度，以使模型适应各种场景。如表(4)所示，RoFormer的准确率随着序列长度上限的增加而提升，这展示了RoFormer处理长文本的能力。我们声称这归因于所提出的RoPE的出色泛化性。

表4：RoFormer 在中文数据集上的预训练策略。训练过程被划分为多个连续阶段。在每个阶段中，我们使用最大序列长度和批量大小的特定组合来训练模型。

Stage	Max seq length	Batch size	Training steps	Loss	Accuracy
1	512	256	200k	1.73	65.0%
2	1536	256	12.5k	1.61	66.8%
3	256	256	120k	1.75	64.6%
4	128	512	80k	1.83	63.4%
5	1536	256	10k	1.58	67.4%
6	512	512	30k	1.66	66.2%

4.5.3 下游任务与数据集

我们选择中国人工智能与法律2019相似案例匹配（CAIL2019-SCM）Xiao等人[2019]数据集来说明RoFormer在处理长文本，即语义文本匹配方面的能力。CAIL2019-SCM包含8964个三元组

2本实验采用来自 https://github.com/lucidrains/performer-pytorch 的代码（MIT 许可证）

最高人民法院发布的部分案例。输入三元组记为(A、B和C)，是三个案件的事实描述。任务是预测在预定义的相似度度量下，对(A, B)是否比(A, C)更接近。需要注意的是，由于文档长度（即大多超过512个字符），现有方法在CAIL2019-SCM数据集上大多无法显著表现。我们按照著名的6:2:2比例划分训练集、验证集和测试集。

4.5.4 结果

我们将预训练的RoFormer模型应用于CAIL2019-SCM，并采用不同的输入长度。该模型与在相同预训练数据上训练的预训练BERT和WoBERT模型进行了比较，如表(5)所示。在短文本截断（即512）的情况下，RoFormer的结果与WoBERT相当，并略优于BERT实现。然而，当将最大输入文本长度增加到1024时，RoFormer以1.5%的绝对提升优于WoBERT。

表5：CAIL2019-SCM任务的实验结果。第一列中的数字表示最大截断序列长度。结果以百分比准确率表示。

Model	Validation	Test
BERT-512	64.13%	67.77%
WoBERT-512	64.07%	68.10%
RoFormer-512	64.13%	68.29%
RoFormer-1024	**66.07**%	**69.79**%

4.5.5 工作的局限性

尽管我们提供了理论依据以及有前景的实验验证，但我们的方法受以下事实的限制：

- 尽管我们在数学上将相对位置关系格式化为二维子空间下的旋转，但缺乏对为何其比采用其他位置编码策略的基线模型收敛更快的透彻解释。
- 尽管我们已经证明我们的模型在内部词元乘积方面具有良好的长期衰减特性（第(3.3)节），这与现有的位置编码机制类似，我们的模型在长文本上表现出优于同类模型的性能，但我们尚未提出一个忠实的解释。

我们提出的RoFormer建立在基于Transformer的基础架构之上，这需要硬件资源来进行预训练。

5 结论

在这项工作中，我们提出了一种新的位置嵌入方法，该方法在自注意力中引入了显式的相对位置依赖关系，以提升Transformer架构的性能。我们的理论分析表明，相对位置可以通过自注意力中的向量积自然地表达，而绝对位置信息则通过旋转矩阵进行编码。此外，我们从数学上说明了所提方法应用于Transformer时的优越性质。最后，在英文和中文基准数据集上的实验表明，我们的方法能够促进预训练中更快的收敛。实验结果还显示，我们提出的RoFormer在长文本任务上能够取得更好的性能。

参考文献

Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, 和 Yann N Dauphin. 卷积序列到序列学习. 在 *International Conference on Machine Learning* 中, 第 1243–1252 页. PMLR, 2017.

Md. Amirul Islam、Sen Jia 和 Neil D. B. Bruce。卷积神经网络编码了多少位置信息？*ArXiv*，abs/2001.08248，2020。

阿希什·瓦斯瓦尼、诺姆·沙泽尔、尼基·帕尔马、雅各布·乌兹科雷特、利昂·琼斯、艾丹·N·戈麦斯、卢卡什·凯泽和伊利亚·波洛苏欣。注意力就是你所需要的一切。在I. Guyon、U. V. Luxburg、S. Bengio、H. Wallach、R. Fergus、S. Vishwanathan和R. Garnett编辑的*Advances in Neural Information Processing Systems*中，

第30卷。Curran Associates, Inc.，2017年。URL https://proceedings.neurips.cc/paper/2017/file/ 3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf。J. Devlin、Ming-Wei Chang、Kenton Lee 和 Kristina Toutanova。Bert：用于语言理解的深度双向变换器预训练。载于 *NAACL-HLT*，2019年。A. Radford、Jeffrey Wu、R. Child、David Luan、Dario Amodei 和 Ilya Sutskever。语言模型是无监督的多任务学习器。2019年。Chulhee Yun、Srinadh Bhojanapalli、Ankit Singh Rawat、Sashank Reddi 和 Sanjiv Kumar。变换器是序列到序列函数的通用逼近器吗？载于 *International Conference on Learning Representations*，2020年。URL https://openreview.net/forum?id=ByxRM0Ntvr。Zhenzhong Lan、Mingda Chen、Sebastian Goodman、Kevin Gimpel、Piyush Sharma 和 Radu Soricut。Albert：一种用于语言表示自监督学习的轻量级 bert。载于 *International Conference on Learning Representations*，2020年。URL https://openreview.net/forum?id=H1eA7AEtvS。Kevin Clark、Minh-Thang Luong、Quoc V. Le 和 Christopher D. Manning。ELECTRA：将文本编码器预训练为判别器而非生成器。载于 *ICLR*，2020年。URL https://openreview.net/pdf?id=r1xMH1BtvB。A. Radford 和 Karthik Narasimhan。通过生成式预训练改进语言理解。2018年。Ankur P. Parikh、Oscar Täckström、Dipanjan Das 和 Jakob Uszkoreit。一种用于自然语言推理的可分解注意力模型。载于 *EMNLP*，2016年。Peter Shaw、Jakob Uszkoreit 和 Ashish Vaswani。具有相对位置表示的自注意力。载于 *NAACL-HLT*，2018年。Cheng-Zhi Anna Huang、Ashish Vaswani、Jakob Uszkoreit、Noam Shazeer、I. Simon、C. Hawthorne、Andrew M. Dai、M. Hoffman、M. Dinculescu 和 D. Eck。音乐变换器。*arXiv: Learning*，2018年。Zihang Dai、Z. Yang、Yiming Yang、J. Carbonell、Quoc V. Le 和 R. Salakhutdinov。Transformer-xl：超越固定长度上下文的注意力语言模型。载于 *ACL*，2019年。Z. Yang、Zihang Dai、Yiming Yang、J. Carbonell、R. Salakhutdinov 和 Quoc V. Le。Xlnet：用于语言理解的广义自回归预训练。载于 *NeurIPS*，2019年。Colin Raffel、Noam Shazeer、Adam Roberts、Katherine Lee、Sharan Narang、Michael Matena、Yanqi Zhou、W. Li 和 Peter J. Liu。使用统一的文本到文本变换器探索迁移学习的极限。*J. Mach. Learn. Res.*，21：140:1–140:67，2020年。Guolin Ke、Di He 和 T. Liu。重新思考语言预训练中的位置编码。*ArXiv*，abs/2006.15595，2020年。Pengcheng He、Xiaodong Liu、Jianfeng Gao 和 Weizhu Chen。Deberta：具有解耦注意力的解码增强型 bert。*ArXiv*，abs/2006.03654，2020年。Zhiheng Huang、Davis Liang、Peng Xu 和 Bing Xiang。使用更好的相对位置嵌入改进变换器模型。载于 *Findings of the Association for Computational Linguistics: EMNLP 2020*，第3327–3335页，Online，2020年11月。计算语言学协会。doi:10.18653/v1/2020.findings-emnlp.298。URL https://www.aclweb.org/anthology/2020.findings-emnlp.298。Xuanqing Liu、Hsiang-Fu Yu、Inderjit S. Dhillon 和 Cho-Jui Hsieh。使用连续动力模型为变换器学习编码位置。载于 *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*，*Proceedings of Machine Learning Research* 第119卷，第6327–6335页。PMLR，2020年。URL http://proceedings.mlr.press/v119/liu20n.html。Tian Qi Chen、Yulia Rubanova、Jesse Bettencourt 和 David Duvenaud。神经常微分方程。载于 Samy Bengio、Hanna M. Wallach、Hugo Larochelle、Kristen Grauman、Nicolò Cesa-Bianchi 和 Roman Garnett 编，*Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*，第6572–6583页，2018a。URL https: //proceedings.neurips.cc/paper/2018/hash/69386f6bb1dfed68692a24c8686939b9-Abstract.html。Benyou Wang、Donghao Zhao、Christina Lioma、Qiuchi Li、Peng Zhang 和 Jakob Grue Simonsen。在复嵌入中编码词序。载于 *International Conference on Learning Representations*，2020年。URL https://openreview.net/forum?id=Hke-WTVtwr。Angelos Katharopoulos、Apoorv Vyas、Nikolaos Pappas 和 François Fleuret。变换器是 rnn：具有线性注意力的快速自回归变换器。载于 *International Conference on Machine Learning*，第5156–5165页。PMLR，2020年。Zhuoran Shen、Mingyuan Zhang、Haiyu Zhao、Shuai Yi 和 Hongsheng Li。高效注意力：具有线性复杂度的注意力。载于 *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*，第3531–3539页，2021年。

Amapreet Singh, Julian Michael, Felix Hill, Omer Levy, 和 Samuel Bowman. Glue: 一个多任务基准和分析平台, 用于自然语言理解. 04 2018. Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, A. Gane , Tamás Sarlós, Peter Hawkins, J. Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy J. Colwell, 和 Adri an Weller. 用performers重新思考注意力. *ArXiv*, abs/2009.14794, 2020. Ondrej Bojar, Christian Buck, Christian Fed ermann, Barry Haddow, Philipp Koehn, Johannes Leveling, Christof Monz, Pavel Pecina, Matt Post, Herve Saint-Am and, Radu Soricut, Lucia Specia, 和 Alevs Tamchyna. 2014年统计机器翻译研讨会的发现. 页码 12–58, 06 2014. d oi:10.3115/v1/W14-3302. Rico Sennrich, Barry Haddow, 和 Alexandra Birch. 使用子词单元进行稀有词的神经机 器翻译. 08 2015. Myle Ott, Sergey Edunov, Alexei Baevski, Angela Fan, Sam Gross, Nathan Ng, David Grangier, 和 Michael Auli. fairseq: 一个快速、可扩展的序列建模工具包. 页码 48–53, 01 2019. doi:10.18653/v1/N19-4009. Kishore Papineni, Salim Roukos, Todd Ward, 和 Wei Jing Zhu. Bleu: 一种机器翻译自动评估方法. 10 2002. doi:10 .3115/1073083.1073135. Yukun Zhu, Ryan Kiros, Richard Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio T orralba, 和 Sanja Fidler. 对齐书籍和电影：通过观看电影和阅读书籍实现故事般的视觉解释. 在 *arXiv preprint arXiv:1506.06724*, 2015. Wikimedia Foundation. Wikimedia下载, https://dumps.wikimedia.org, 2021. Ilya Loshchilov 和 Frank Hutter. 解耦权重衰减正则化. *arXiv e-prints*, 文章 arXiv:1711.05101, 十一月 2017. William B. Dolan 和 Chris Brockett. 自动构建句子释义语料库. 在 *Proceed-
ings of the Third International Workshop on Paraphrasing (IWP2005)*, 2005. URL https://www.aclweb.org/ antholo gy/I05-5002. Richard Socher, A. Perelygin, J.Y. Wu, J. Chuang, C.D. Manning, A.Y. Ng, 和 C. Potts. 用于情感树库 语义组合性的递归深度模型. *EMNLP*, 1631:1631–1642, 01 2013. Pranav Rajpurkar, Jian Zhang, Konstantin Lopyr ev, 和 Percy Liang. Squad: 100,000+ 个用于文本机器理解的问题. 页码 2383–2392, 01 2016. doi:10.18653/v1/D1 6-1264. Hussein Al-Natsheh. Udl在semeval-2017任务1：使用成对特征上的回归模型估计英语句子对的语义文 本相似度. 08 2017. Z. Chen, H. Zhang, 和 L. Zhang, X.and Zhao. Quora问题对., 2018b. URL https://www.kaggle.c om/c/ quora-question-pairs. Adina Williams, Nikita Nangia, 和 Samuel Bowman. 一个广泛覆盖的挑战语料库，用 于通过推理进行句子理解. 页码 1112–1122, 01 2018. doi:10.18653/v1/N18-1101. Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Fun towicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le S cao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, 和 Alexander M. Rush. Transformers: 最先进的自然语言处 理. 在 *Proceedings of the 2020 Conference on Empirical
Methods in Natural Language Processing: System Demonstrations*, 页码 38–45, Online, 十月 2020. 计算语言学协会 . URL https://www.aclweb.org/anthology/2020.emnlp-demos.6. Matt Mahoney. 大型文本压缩基准, http://www.mat tmahoney.net/dc/text.html, 2006. Jianlin Su. Wobert: 基于词的中文bert模型 - zhuiyiai. 技术报告, 2020. URL https: //github.com/ ZhuiyiTechnology/WoBERT. Victor Junqiu Wei, Xiaozhe Ren, Xiaoguang Li, Wenyong Huang, Yi Li ao, Yasheng Wang, Jiashu Lin, Xin Jiang, Xiao Chen, 和 Qun Liu. Nezha: 用于中文语言理解的神经上下文化表示 . 08 2019. Chaojun Xiao, Haoxi Zhong, Zhipeng Guo, Cunchao Tu, Zhiyuan Liu, Maosong Sun, Tianyang Zhang, Xi anpei Han, Zhen hu, Heng Wang, 和 Jianfeng Xu. Cail2019-scm: 法律领域相似案例匹配数据集. 11 2019.